

Wdrażanie aplikacji

CRC 2026

michal.bzowski@ing.com

Wstęp

- Najważniejsze pojęcia:
 - Artefakt
 - Deploy (Wdrożenie) vs Release (Wydanie)
 - Serwer vs Host
 - Strategie wdrażania i releasowania aplikacji
- Gdzie możemy wdrożyć aplikację?
- Dobre praktyki (<https://12factor.net/>)

Artefakt

Czym jest artefakt?

- To gotowy rezultat procesu budowania, który można wdrożyć na środowisko docelowe.
- W świecie Javy artefaktem jest najczęściej archiwum ZIP z określoną strukturą -> **JAR**

JAR (Java ARchive)

Podstawowy artefakt Javy zawiera:

- skompilowane klasy (.class)
- zasoby (.properties, .yml)
- metadane (MANIFEST.MF)

Jak jest uruchamiany:

```
java -jar app.jar
```

Typowe użycie:

- aplikacje standalone
- Spring Boot
- batch, CLI, mikroserwisy
- Fat / Uber JAR
 - zawiera wszystkie zależności
 - nie wymaga serwera aplikacyjnego

Zalety:

- ✓ prostota
- ✓ szybki deploy
- ✓ pełna kontrola

Wady:

✗ aplikacja sama musi ogarniać:

- security
- transakcje
- connection pooling

📌 Spring Boot sprawił, że JAR stał się standardem wdrożeń.

📌 Inne rodzaje artefaktów: później

Wdrożenie

- Skrócona definicja:
 - Wdrożenie aplikacji to proces przygotowania, uruchomienia i formalnego przekazania systemu do rzeczywistego użycia, obejmujący aspekty techniczne, organizacyjne, prawne i operacyjne.
 - Wgranie aplikacji na serwer jest jedynie jednym z technicznych kroków tego procesu.

Wdrożenie: Przygotowanie

- Określenie zakresu funkcjonalnego
- Ustalenie odpowiedzialności (właściciel systemu, administrator, wsparcie)
- Kwestie umowne:
 - Umowa wdrożeniowa
 - SLA (Service Level Agreement)
 - Licencje
 - Ustalenie kryteriów odbioru (acceptance criteria)

Wdrożenie: Środowiska (1/2)

Wdrożenie zakłada świadome rozróżnienie środowisk, np.:

- DEV – środowisko deweloperskie
- TEST / QA – testy integracyjne i akceptacyjne
- UAT (Acceptance) – testy użytkownika biznesowego
- PROD – środowisko produkcyjne

Inny podział: Dev, Staging, Production

Wdrożenie: Środowiska (2/2)

Dla każdego środowiska przygotowuje się:

- Infrastrukturę (serwery, chmura, kontenery)
- Konfigurację
- Dostęp do danych
- Polityki bezpieczeństwa

Wdrożenie: Przygotowanie techniczne

- Budowa artefaktu aplikacji
- Konfiguracja środowiskowa
- Integracja z innymi systemami
- Migracja danych (jeśli dotyczy)
- Konfiguracja monitoringu i logów
- Kopie zapasowe (backup)

Uruchomienie aplikacji na środowisku docelowym

Dopiero tutaj pojawia się to, co potocznie nazywamy „wdrożeniem”, czyli:

- Instalacja / uruchomienie aplikacji
- Start na środowisku produkcyjnym
- Testy powdrożeniowe (smoke tests)
- Jest to jeden z etapów wdrożenia, a nie całe wdrożenie.

Wdrożenie cd.

- testy akceptacyjne
- podpisanie protokołu odbioru
- szkolenie użytkowników
- przekazanie systemu do eksploatacji

Utrzymanie powdrożeniowe

Często formalnie to nadal część wdrożenia:

- wsparcie powdrożeniowe
- poprawki
- monitoring stabilności
- reagowanie na błędy

Deploy

- Tłumaczone jako "Wdrożenie"
- W praktyce programisty / DevOps'a
 - To fizyczne przeniesienie artefaktu na serwer
 - I inne czynności wymagane przez biznes

Różnica między Wdrożeniem a Wdrożeniem (deploy'em)

- Pod jednym kryje się cały proces
- Pod drugim "prosta czynność" przeniesienia artefaktu

Release

- Tłumaczone jako "Wydanie"
 - Udostępnienie funkcjonalności klientowi końcowemu
- ? Czy jest różnica między Deploy a Release?

Release vs Deploy – na czym polega różnica?

- W skrócie
 - Deploy = techniczne wgranie aplikacji
 - Release = udostępnienie funkcjonalności użytkownikom
- Możesz:
 - Zdeployować bez release
 - Release'ować bez nowego deploya
 - Zrobić jedno i drugie naraz

Strategie Deploy / Release

- Poprosto wrzucenie aplikacji (stop starej / start nowej)
- Feature Flags / Feature Toggles (Przełączniki Funkcji)
- Canary release
- Blue / Green Deployment
- Shadow Deployment / Dark Launching (Wdrożenie w Cieniu)
- A/B Testing Deployment

Gdzie można wdrożyć aplikację?

? No gdzie?

Gdzie można wdrożyć aplikację?

- Aplikację zawsze wdrażamy na jakąś maszynę (host),
- a czasem dodatkowo uruchamiamy ją wewnątrz serwera aplikacyjnego, kontenera albo platformy chmurowej.

Host – gdzie aplikacja fizycznie działa

- Fizyczna maszyna (bare metal)
- VPS
- Maszyna wirtualna (VM)
- Z zainstalowanym systemem operacyjnym (najczęściej Linux)

Host – gdzie aplikacja fizycznie działa

Charakterystyka:

- system operacyjny
- użytkowników
- procesy
- sieć, porty, firewall
- aplikacja Java działa jako zwykły proces OS

```
java -jar app.jar
```

Host – gdzie aplikacja fizycznie działa

Zalety:

- ✓ pełna kontrola
- ✓ proste do zrozumienia
- ✓ dobre na start / naukę

Wady:

- ✗ ręczne zarządzanie
- ✗ skalowanie boli
- ✗ „works on my server”

Serwer aplikacyjny

- JBoss / WildFly
- WebLogic
- WebSphere
- Tomcat

Serwer aplikacyjny

- to oprogramowanie, które:
 - uruchamia aplikacje
 - zarządza ich cyklem życia
 - dostarcza gotowe mechanizmy:
 - security
 - transakcje
 - connection pool
 - JNDI
 - Kolejki

Serwer aplikacyjny

```
Host (Linux)
├─ JVM
│   └─ Application Server
│       └─ Aplikacja (WAR / EAR)
```

Serwer aplikacyjny

Zalety:

- ✓ standardy enterprise
- ✓ centralne zarządzanie
- ✓ znane w dużych korporacjach

Wady:

- ✗ ciężkie
- ✗ wolniejsze wdrożenia
- ✗ mniejsza elastyczność

Kontenery – nowoczesny standard

Czym są kontenery:

aplikacja + runtime + konfiguracja
uruchamiana w izolowanym środowisku
nadal na hoście, ale w kontrolowany sposób

Kontenery - nowoczesny standard

```
Host (Linux)
└─ Docker
    └─ Kontener
        └─ JVM + App
```

Kontenery - nowoczesny standard

Wdrażamy obraz kontenera, nie JAR - konfiguracja przez zmienne środowiskowe

Zalety:

- ✓ powtarzalność
- ✓ szybkie wdrożenia
- ✓ łatwe skalowanie

Wady:

- ✗ dodatkowa warstwa złożoności
- ✗ trzeba zrozumieć Docker / networking

Spring Boot wziął ideę serwera aplikacyjnego i „schował go do środka aplikacji”

Kontener to standardowy sposób dostarczania aplikacji, niezależnie od środowiska.

**Chmura – „ktoś zarządza
infrastrukturą za nas”**

IaaS – Infrastructure as a Service

Co dostajemy:

maszynę (VM)

system operacyjny

sieć, dyski

IaaS – Infrastructure as a Service

Co robimy sami:

instalujemy Javę

konfigurujemy serwer

wdrażamy aplikację

IaaS – Infrastructure as a Service

Przykłady:

AWS EC2

Azure Virtual Machines

GCP Compute Engine

IaaS – Infrastructure as a Service

IaaS = „host w cudzym data center”

PaaS – Platform as a Service

Co dostajemy:

gotowe środowisko do uruchamiania aplikacji
bez zarządzania OS

PaaS – Platform as a Service

Co robimy:

wrzucamy aplikację
konfigurujemy parametry

Przykłady:

Azure App Service

Heroku

Google App Engine

PaaS – Platform as a Service

 Model:

```
Chmura
├── Platforma
│   └── Aplikacja
```

PaaS – Platform as a Service

Zalety:

- ✓ szybki start
- ✓ brak administrowania
- ✗ mniejsza kontrola

Serverless – bez serwera (z punktu widzenia developera)

Co to naprawdę znaczy:

serwery istnieją
my się nimi nie zajmujemy

Serverless – bez serwera

Model:

funkcje uruchamiane „na żądanie”
płacimy za wykonanie, nie za uptime

Przykłady:

AWS Lambda

Azure Functions

Serverless – bez serwera

Charakterystyka:

- ✓ zero zarządzania
- ✓ dobra do eventów / API
- ✗ nie każda aplikacja się nadaje

 Ważne zdanie:

Serverless nie oznacza „bez serwera”, tylko „bez odpowiedzialności za serwer”

Podsumowanie - gdzie wdrażamy

Opcja	Kto zarządza OS	Kontrola	Złożoność	Typowe użycie
Host	My	wysoka	niska	nauka, małe projekty
Serwer app	My	wysoka	średnia	enterprise legacy
Kontenery	My	wysoka	średnia	standard rynkowy
IaaS	My	wysoka	średnia	elastyczne systemy
PaaS	Chmura	średnia	niska	szybki development
Serverless	Chmura	niska	niska	event-driven, API, zadania batch

Artefakty - ciąg dalszy

WAR (Web ARchive)

Co to jest:

artefakt aplikacji webowej Java

nie działa samodzielnie

wymaga serwera aplikacyjnego lub servlet container

Co zawiera:

klasy aplikacji

pliki webowe (HTML, JSP)

konfigurację webową

WAR (Web ARchive)

Jak jest wdrażany:

kopiowany do serwera:

Tomcat

JBoss / WildFly

WebLogic

WAR (Web ARchive)

Model uruchomienia:

```
Host
└─ JVM
    └─ Application Server
        └─ app.war
```


WAR (Web ARchive)

Zalety:

- ✓ separacja aplikacji i runtime
- ✓ centralne zarządzanie
- ✓ standard JEE

Wady:

- ✗ wolniejszy deploy
- ✗ zależność od serwera
- ✗ trudniejsze skalowanie

WAR był standardem w świecie enterprise, zanim pojawił się Spring Boot.

EAR (Enterprise ARchive)

Co to jest:

kontener dla wielu modułów
agreguje kilka aplikacji w jeden pakiet

Co może zawierać:

WAR (frontend)

JAR (logika biznesowa)

EJB

wspólne biblioteki

EAR (Enterprise ARchive)

Model:

```
EAR
├── web.war
├── business.jar
└── common.jar
```

EAR (Enterprise ARchive)

Gdzie używany:

duże systemy korporacyjne
monolity enterprise
starsze architektury JEE

EAR (Enterprise ARchive)

Model uruchomienia:

```
Host
├── JVM
│   ├── Enterprise App Server
│   │   └── app.ear
```

EAR (Enterprise ARchive)

Zalety:

- ✓ modularność
- ✓ centralne zarządzanie
- ✓ integracja wielu komponentów

Wady:

- ✗ wysoka złożoność
- ✗ długi czas wdrożeń
- ✗ mała elastyczność
- 📌 EAR to „kontener na aplikacje”, a nie pojedyncza aplikacja.

The Twelve Factors

- <https://12factor.net/>
- Zestaw 12 dobrych praktyk i zasad projektowania nowoczesnych aplikacji typu SaaS (Software as a Service).
- Powstały (zostały zaobserwowane i spisane), aby ułatwić tworzenie oprogramowania, które jest skalowalne, przenośne i łatwe we wdrażaniu, zwłaszcza w środowiskach chmurowych.

1. Codebase – jeden codebase, wiele wdrożeń

- Jedno repozytorium kodu, wiele środowisk (dev / test / prod)

W Spring Boot:

- jedno repo Git
- różne środowiska uruchomieniowe
- zero forków kodu

Środowisko nie zmienia kodu, tylko konfigurację.

2. Dependencies – zależności jawnie deklarowane

Aplikacja deklaruje wszystkie zależności explicite

- pom.xml / build.gradle
- brak „magicznych bibliotek” na serwerze

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-web</artifactId>  
</dependency>
```

3. Config – konfiguracja poza kodem

Konfiguracja NIE znajduje się w kodzie

Używamy:

- zmienne środowiskowe
- profile
- application.yml jako fallback

4. Backing services – usługi zewnętrzne jako zasoby

Baza, cache, kolejka = zasób zewnętrzny

PostgreSQL / Redis / RabbitMQ

Spring Boot traktuje je jak konfigurację można je podmienić bez zmiany kodu

Zmiana bazy $\neq$ zmiana kodu.

5. Build, Release, Run – trzy osobne etapy

Rozdzielamy budowę / wydanie / uruchomienie

- Jeden artefakt może przejść kilka środowisk, zanim trafi na produkcję.
- Wydanie może być odroczone w czasie po budowie.
- Uruchomienie w razie awarii jest oderwane od budowy i wydania.

6. Processes – aplikacja jako proces bezstanowy

Aplikacja jest stateless

Nie trzymamy stanów pamięci lub w sesji HTTP

Stan ma być w

- DB
- Cache
- Storage

7. Port binding – aplikacja sama nasłuchuje

Aplikacja wystawia port sama

- Nie potrzebujemy Apache / Nginx do startu
- Idealne do kontenerów i chmury

```
server:  
  port: ${PORT:8080}
```

```
export PORT=7999
```

8. Concurrency – skalowanie przez procesy

Skalujemy przez uruchamianie kolejnych instancji

W praktyce:

- kilka instancji aplikacji
- load balancer
- Docker / Kubernetes

Nie zwiększamy mocy jednej instancji – zwiększamy ich liczbę.

9. Disposability – szybki start i shutdown

Aplikacja powinna szybko się uruchamiać i zamykać

Spring Boot:

- szybki startup
- poprawne zamykanie kontekstu
- reakcja na SIGTERM

Idealne do: Docker, Kubernetes, autoscaling

10. Dev/Prod parity – podobne środowiska

Dev $\approx$ Prod

DTAP vs Dev / Stage / Production

Im większa różnica między dev a prod, tym więcej niespodzianek.

11. Logs – logi jako strumień zdarzeń

Aplikacja nie zarządza logami

- logi na stdout
- brak plików logów w aplikacji

Docker / chmura:

- Zbiera logi
- Agreguje
- Analizuje

12. Admin processes – zadania administracyjne

Zadania admin uruchamiane jak normalny proces

Przykład:

- migracje DB
- batch job
- cleanup

Przykład praktyczny 1

```
cd crc-2026-deployments
mvn clean package
docker compose build
docker compose up -d
Sprawdź http://localhost - odśwież kilkukrotnie
Zmień nginx/nginx.conf - zakomentuj linię #server linux1:8080;
docker exec nginx-lb nginx -s reload
docker stop linux1
mvn clean package
docker compose build linux1
docker compose up -d linux1
```